SmartCopy
Interface Control Document

Version A

J. Dowell

February 2026

Approved:

System Engineering

Date: _____

Approved:

System Architecture

Date: _____

Long Wavelength Array Project

Change Record

Version	Date	Author(s)	Changes
A	February 2026	J. Dowell	Initial release

Contents

Change Record **1**

List of Tables **5**

1 Description **6**
1.1 Purpose . . . 6
1.2 Scope . . . 6
1.3 System Overview . . . 7
1.4 Key Functions . . . 7
1.4.1 Command Processing . . . 7
1.4.2 File Transfer . . . 8
1.4.3 Source File Purging . . . 8
1.4.4 Failure Notification . . . 8
1.5 Applicable Stations . . . 8

2 Abbreviations and Definitions **9**
2.1 Abbreviations . . . 9
2.2 Definitions . . . 10
2.3 Document Conventions . . . 10

3 Monitor and Control Interface **11**
3.1 UDP Command Protocol . . . 11
3.1.1 Network Configuration . . . 11
3.1.2 Packet Structure . . . 12
3.1.3 Response Format . . . 12
3.1.4 Reference ID Server . . . 12
3.2 System Commands . . . 13
3.2.1 PNG — Ping . . . 13
3.2.2 INI — Initialize . . . 13
3.2.3 SHT — Shutdown . . . 13
3.2.4 RPT — Report . . . 14
3.3 Data Management Commands . . . 14
3.3.1 SCP — Smart Copy . . . 14
3.3.2 SRM — Smart Remove . . . 14
3.3.3 PAU — Pause . . . 15
3.3.4 RES — Resume . . . 15
3.3.5 SCN — Smart Cancel . . . 15

3.4 MIB Entries . . . 16
3.4.1 Per-DR Observing Status . . . 16
3.4.2 Per-DR Queue Status . . . 16
3.4.3 Per-DR Active Copy Status . . . 16

4 Queue Management **18**
4.1 Queue Architecture . . . 18
4.2 Database Schema . . . 18
4.3 Item Status Lifecycle . . . 18
4.4 Retry Logic . . . 20
4.5 Remote Copy Exclusivity . . . 20
4.6 Delete Markers . . . 21
4.7 Purge Operations . . . 21

5 Copy Engine **22**
5.1 Overview . . . 22
5.2 Copy Modes . . . 22
5.2.1 Local to Local Copy . . . 22
5.2.2 Local to Remote Copy . . . 23
5.2.3 Remote Same-Host Copy . . . 23
5.2.4 Remote to Remote Copy . . . 23
5.3 Pause and Resume . . . 23
5.4 Destination File Truncation . . . 23
5.5 Bandwidth Limiting . . . 24
5.5.1 Gateway Detection . . . 24
5.6 Progress Monitoring . . . 24
5.7 Delete Operations . . . 25
5.8 Copy Lifecycle . . . 25

6 Station Monitoring **26**
6.1 MCS Log Monitoring . . . 26
6.2 Log Entry Parsing . . . 26
6.3 State Determination . . . 26
6.4 State Change Propagation . . . 27
6.5 Initialization Behavior . . . 27

7 Status and Error Handling **28**
7.1 System States . . . 28
7.2 State Transitions . . . 28
7.2.1 State Transition Events . . . 28
7.3 Command Exit Codes . . . 28
7.4 Error Response Format . . . 29
7.5 Process Locking . . . 30
7.6 Email Notifications . . . 30

8 References **31**
8.1 Related Documents . . . 31
8.2 Software Repository . . . 31

8.3 Configuration Files 32
8.4 Dependencies 32
8.5 External Tools 33
8.6 Service Deployment 33

A Exit and Status Code Reference 34
A.1 Command Exit Codes 34
A.2 System Status Values 34
A.3 Response Code Characters 35
A.4 Queue Item Statuses 35
A.5 Copy Operation Statuses 36
A.6 Error Response Format 36

List of Tables

3.1 Default Network Ports 11
3.2 Command Packet Structure 12
3.3 Response Packet Structure 12
3.4 System Commands 13
3.5 Data Management Commands 14
3.6 General MIB Entries 16
3.7 Observing Status MIB Entry 16
3.8 Queue MIB Entries 16
3.9 Active Copy MIB Entries 17

4.1 Queue Database Schema (`queue_items` Table) 19
4.2 Queue Item Statuses 19

5.1 Copy Modes 22

6.1 Parsed Log Entry Fields 26
6.2 DR State Determination Rules 27

7.1 System Status Values 28
7.2 Command Exit Codes 29

8.1 SmartCopy Source Files 31
8.2 Software Dependencies 32
8.3 Required System Tools 33

A.3 Response Code Characters 35

Section 1

Description

1.1 Purpose

This document defines the interface control specifications for the SmartCopy system. SmartCopy is an autonomous data management system that runs on the MCS task processor at LWA stations, providing intelligent file copying and deletion services for the Data Recorders (DRs). Its primary functions include:

- Queued file copying from DRs to local and remote destinations, e.g., the LWA UCF
- Queued file deletion with configurable purge thresholds
- Per-DR queue management with pause and resume capabilities
- Automatic retry of failed transfers with configurable backoff
- Bandwidth-limited remote transfers
- Station monitoring to avoid copying during active observations

This ICD serves as the authoritative reference for developers, integrators, and operators working with the SmartCopy system.

1.2 Scope

This document covers the following aspects of the SmartCopy system:

1. **Command Interface**: UDP-based command protocol for communicating with SmartCopy, including system commands, copy/delete commands, and queue management commands.
2. **Queue Management**: Per-DR persistent queuing system backed by SQLite, including queue operations, retry logic, and automatic purging.
3. **Copy Engine**: The `rsync`-based file transfer engine, including local copies, remote copies via SSH, bandwidth limiting, pause/resume, and destination file truncation.
4. **Station Monitoring**: Monitoring of the MCS log file to track DR busy/idle states and avoid copying during active recording. Because SmartCopy operates independently from MCS, reading the log file is its only mechanism for observing station activity.

5. **Status and Error Handling**: System state management, exit codes, error reporting, and email notifications.

1.3 System Overview

SmartCopy runs on the MCS task processor at LWA stations and communicates with the DRs via SSH for file operations. Although SmartCopy uses the same network infrastructure as MCS and other LWA subsystems, it operates independently: MCS does not see or route SmartCopy messages, and SmartCopy cannot see MCS or subsystem messages. This separation is why SmartCopy must monitor the MCS log file to track DR state (see Section 6). The system receives commands via UDP and manages independent copy queues for each DR. The system architecture consists of:

Command Handler Receives and processes UDP commands using a packet format based on the MCS packet protocol. The command handler listens for incoming packets, dispatches commands to the appropriate functions, and returns responses.

Reference Server A ØMQ REP socket that serves unique reference IDs for command packets. The reference ID counter is periodically saved to disk and survives restarts.

Station Monitor A background thread that tails the MCS `mselog.txt` log file to track DR busy/idle states. Because SmartCopy cannot directly observe MCS traffic, the log file is its only source of station activity information. When a DR transitions to idle, its copy queue is automatically resumed; when it becomes busy (recording), the queue is paused.

DR Queue Managers One background thread per DR that processes the persistent copy queue. Each manager pulls items from the queue, executes copy or delete operations, handles retries, and manages completed/failed file tracking.

Copy Engine The `InterruptibleCopy` class that wraps `rsync` for file transfers. Supports local copies, remote copies via SSH, bandwidth limiting, pause/resume via process signals, and progress tracking.

1.4 Key Functions

SmartCopy performs the following primary functions:

1.4.1 Command Processing

SmartCopy accepts commands via UDP packets:

- `PNG`: Ping/heartbeat verification
- `INI`: System initialization
- `SHT`: System shutdown
- `SCP`: Queue a file copy operation
- `SRM`: Queue a file delete operation
- `PAU`: Pause a DR copy queue
- `RES`: Resume a DR copy queue

- `SCN`: Cancel a queued copy
- `RPT`: Report MIB (Management Information Base) entries

1.4.2 File Transfer

When a copy command is received, SmartCopy:

- Assigns a unique ID to the copy operation
- Adds the operation to the appropriate DR's persistent queue
- Waits for the DR to become idle before starting the transfer
- Executes the transfer using `rsync` with progress monitoring
- Tracks completed files for later source deletion
- Retries failed transfers with configurable wait intervals

1.4.3 Source File Purging

SmartCopy manages source file cleanup:

- Tracks completed transfers and their file sizes
- When accumulated completed data exceeds the `purge_size` threshold (default: 1 TB), deletes source files from the DR
- Purge runs are performed daily at 18:00 UTC
- Spectrometer files are only eligible for deletion after being copied to the remote archive

1.4.4 Failure Notification

SmartCopy sends email notifications for failed transfers:

- Failed file reports are generated daily alongside purge operations
- Reports include file paths, sizes, and failure reasons
- Notifications are sent to the operations mailing list via SMTP

1.5 Applicable Stations

This ICD applies to all LWA stations running the SmartCopy system. Station-specific configuration parameters are defined in the `defaults.json` configuration file.

Section 2

Abbreviations and Definitions

2.1 Abbreviations

Abbreviation	Definition
DR	Data Recorder - recording subsystem running the DROS software
DRSU	Data Recorder Storage Unit - removable storage for DRs
FIFO	First In, First Out - queue processing order
ICD	Interface Control Document
LWA	Long Wavelength Array
LWA1	Long Wavelength Array station 1
LWA-NA	Long Wavelength Array North Arm station
LWA-SV	Long Wavelength Array Sevilleta station
MCS	Monitor and Control System
MIB	Management Information Base - hierarchical data structure for status reporting
MJD	Modified Julian Date
MPM	Milliseconds Past Midnight
SMTP	Simple Mail Transfer Protocol
SSH	Secure Shell
UCF	LWA User Computing Facility - cluster for reducing LWA data
UDP	User Datagram Protocol
UTC	Coordinated Universal Time
WAL	Write-Ahead Logging - SQLite journaling mode for reliability
ØMQ	ØMQ (ZeroMQ) - high-performance asynchronous messaging library

2.2 Definitions

Bandwidth Limit A configurable limit on the transfer rate for remote copies, specified in MB/s. A value of 0 disables limiting.

Copy Queue A persistent, per-DR FIFO queue of file transfer operations backed by SQLite. Items survive system restarts.

Global Inhibit A per-DR flag that prevents queue processing regardless of DR state. Set by `PAU` commands and cleared by `RES` commands.

Local Copy A file transfer where the source and destination are on the same host, performed via `rsync` without SSH.

Purge The process of deleting source files from a DR after they have been successfully copied to a destination.

Reference ID A unique integer identifier assigned to each command packet, generated by the ØMQ reference server. IDs range from 1 to 999,999,999 and roll over.

Remote Copy A file transfer where the source and destination are on different hosts, performed via `rsync` over SSH. Only one remote copy may be active at a time across all DR queues.

2.3 Document Conventions

- Command names are shown in `monospace` font (e.g., `PNG`, `SCP`)
- MIB entry names are shown in `monospace` font (e.g., `SUMMARY`)
- Hexadecimal values are prefixed with `0x` (e.g., `0x00`, `0x04`)
- File paths are shown in `monospace` font (e.g., `defaults.json`)
- Code identifiers are shown in `monospace` font (e.g., `InterruptibleCopy`)

Section 3

Monitor and Control Interface

This section defines the command interface for SmartCopy, including the UDP packet protocol, system commands, data management commands, and MIB entries.

Note

SmartCopy uses a UDP packet format based on the MCS standard and shares the same network infrastructure as other LWA subsystems, but operates independently. Like HAL, MCS does not see or route SmartCopy messages, and SmartCopy cannot see MCS or subsystem messages. However, unlike HAL, SmartCopy packets are not cryptographically signed.

3.1 UDP Command Protocol

SmartCopy uses UDP packets for its command interface, following a packet format based on the MCS standard.

3.1.1 Network Configuration

The default network port assignments are listed in Table 3.1.

Table 3.1: Default Network Ports

Port	Default	Description
Receive Port	5050	Incoming commands to SmartCopy
Send Port	5051	Outgoing responses from SmartCopy
Reference Port	5052	ØMQ REP socket for reference IDs

Port assignments are configurable via the `defaults.json` configuration file using the `mcs.message_in_port`, `mcs.message_out_port`, and `mcs.message_ref_port` parameters.

The server bind address is automatically determined by scanning network interfaces for a `10.1.x.0` network. The bind address is set to `10.1.x.2` on the matching subnet.

3.1.2 Packet Structure

Command packets use the following structure, based on the MCS packet format (Table 3.2).

Table 3.2: Command Packet Structure

Bytes	Length	Field	Description
0–2	3	Destination	Subsystem identifier ("SCM")
3–5	3	Sender	Source subsystem identifier
6–8	3	Command	Three-letter command code
9–17	9	Reference	Reference number (zero-padded integer)
18–21	4	Data Length	Length of data section
22–27	6	MJD	Modified Julian Date
28–36	9	MPM	Milliseconds Past Midnight
37	1	-	Space
38+	Variable	Data	Command-specific data payload

3.1.3 Response Format

Response packets follow a similar structure (Table 3.3).

Table 3.3: Response Packet Structure

Bytes	Length	Field	Description
0–2	3	Destination	Original sender
3–5	3	Sender	"SCM"
6–8	3	Command	Echoed from request
9–17	9	Reference	Echoed from request
18–21	4	Data Length	Response data length + 8
22–27	6	MJD	Current timestamp
28–36	9	MPM	Current timestamp
37	1	-	Space
38	1	Response Code	`A` = Accepted, `R` = Rejected
39–45	7	Status	System status string
46+	Variable	Data	Response data

3.1.4 Reference ID Server

SmartCopy provides a ØMQ REP socket on the reference port (default: 5052) for generating unique reference IDs. Clients send the message `next_ref` and receive a unique integer in response. Reference IDs:

- Range from 1 to 999,999,999
- Roll over to 1 when the maximum is reached
- Are persisted to the file `.sc_reference_id` every 10 IDs
- Resume from the persisted value (plus 10) after a restart

3.2 System Commands

The system commands are listed in Table 3.4.

Table 3.4: System Commands

Command	Description	Parameters
`PNG`	Ping/heartbeat verification	None
`INI`	Initialize SmartCopy system	None
`SHT`	Shutdown SmartCopy system	None
`RPT`	Report MIB entry value	MIB key string

3.2.1 PNG — Ping

The `PNG` command verifies system responsiveness. No parameters are required. Returns an accepted response if the system is operational.

3.2.2 INI — Initialize

The `INI` command initializes the SmartCopy system:

- Transitions system state to `BOOTING`
- Creates or restarts the station monitoring thread
- Creates or restarts the per-DR queue manager threads (DR1–DR5)
- Restores pending queue items from the SQLite database
- Clears the global inhibit flag for all DR copy queues
- Transitions system state to `NORMAL` upon completion

The initialization process runs asynchronously. The command is rejected with exit code `0x03` if another `INI` or `SHT` operation is already in progress.

3.2.3 SHT — Shutdown

The `SHT` command shuts down the SmartCopy system:

- Transitions system state to `SHUTDWN` immediately
- Pauses all DR copy queues
- Stops all DR queue manager threads
- Stops the station monitoring thread

The shutdown process runs asynchronously. The command is rejected with exit code `0x03` if another `INI` or `SHT` operation is already in progress. Pending queue items are preserved in the SQLite database and will be restored on the next `INI`.

3.2.4 RPT — Report

The `RPT` command queries MIB entries. The data field contains the MIB key string to query. See Section 3.4 for the full list of queryable entries.

3.3 Data Management Commands

The data management commands are listed in Table 3.5.

Table 3.5: Data Management Commands

Command	Description	Parameters
`SCP`	Queue a file copy	`host:path->dest:path`
`SRM`	Queue a file delete	`[-tNOW] host:path`
`PAU`	Pause a DR queue	`DR{1-5}` or `ALL`
`RES`	Resume a DR queue	`DR{1-5}` or `ALL`
`SCN`	Cancel a queued copy	Copy ID string

3.3.1 SCP — Smart Copy

The `SCP` command queues a file copy operation. The data field contains the source and destination in the format:

```
host:path->dest:path
```

Where *host* is the source hostname (or DR name), *path* is the full source file path, *dest* is the destination hostname, and the second *path* is the destination path. Colons in file paths must be escaped with a backslash.

On success, the response data contains the assigned copy ID. The DR is determined from the *host* field. The command is rejected with:

- `0x04` if the system is not initialized
- `0x01` if the DR name is not recognized
- `0x02` if an error occurs while queuing the copy

3.3.2 SRM — Smart Remove

The `SRM` command queues a file delete operation. The data field format is:

```
[-tNOW] host:path
```

Where *host* is the hostname and *path* is the file to delete. The optional `-tNOW` flag causes the deletion to be executed immediately rather than deferred to the next purge cycle. Without `-tNOW`, the file is marked for deletion but only removed during the daily purge.

On success, the response data contains the assigned delete ID. The command is rejected with:

- `0x04` if the system is not initialized

- `0x01` if the DR name is not recognized
- `0x02` if an error occurs while queuing the delete

3.3.3 PAU — Pause

The `PAU` command pauses copy queue processing for a specified DR. The data field contains either a specific DR name (e.g., `DR1`) or `ALL` to pause all queues. When paused:

- The currently active copy is interrupted (the `rsync` process is killed)
- No new items are pulled from the queue
- The global inhibit flag is set, preventing the station monitor from auto-resuming the queue

The command is rejected with:

- `0x04` if the system is not initialized
- `0x01` if the DR name is not recognized
- `0x02` if an error occurs while pausing the queue

3.3.4 RES — Resume

The `RES` command resumes copy queue processing for a specified DR. The data field contains either a specific DR name or `ALL`. When resumed:

- The currently paused copy is restarted (using `rsync --append` for seamless continuation)
- Queue processing resumes from where it was paused
- The global inhibit flag is cleared, allowing the station monitor to manage the queue

The command is rejected with:

- `0x04` if the system is not initialized
- `0x01` if the DR name is not recognized
- `0x02` if an error occurs while resuming the queue

3.3.5 SCN — Smart Cancel

The `SCN` command cancels a previously queued copy or delete operation. The data field contains the copy ID string returned by the `SCP` or `SRM` command. If the specified copy is currently active, it is stopped. The item is marked as canceled in the results cache and will be skipped when encountered in the queue.

The command is rejected with:

- `0x04` if the system is not initialized
- `0x01` if the copy ID is not found
- `0x02` if an internal error occurs

3.4 MIB Entries

The MIB entries listed in Table 3.6 are queryable via the `RPT` command. All MIB values are returned as ASCII text strings.

Table 3.6: General MIB Entries

Entry	Description
`SUMMARY`	Current system status (7-character string)
`INFO`	Informational message (up to 256 characters)
`LASTLOG`	Last log entry (up to 256 characters)
`SUBSYSTEM`	Subsystem identifier ("SCM")
`SERIALNO`	Serial number
`VERSION`	Software version string

3.4.1 Per-DR Observing Status

The observing status of each DR is available via entries of the form `OBSSTATUS_{DR}` (Table 3.7).

Table 3.7: Observing Status MIB Entry

Entry	Description
`OBSSTATUS_DR{n}`	Recording state of DR{n} ("busy" or "idle")

Example: `OBSSTATUS_DR3` returns the recording state of DR3.

3.4.2 Per-DR Queue Status

Queue information for each DR is available via entries of the form `QUEUE_{property}_{DR}` (Table 3.8).

Table 3.8: Queue MIB Entries

Entry	Description
`QUEUE_SIZE_DR{n}`	Number of pending items in the queue (including active copy)
`QUEUE_STATUS_DR{n}`	Queue state ("active", "paused", or "global paused")
`QUEUE_STATS_DR{n}`	Queue statistics: pending, processing, completed, and failed counts
`QUEUE_ENTRY_{id}`	Status of a specific copy command by ID

Example: `QUEUE_SIZE_DR1` returns the number of items queued for DR1.

3.4.3 Per-DR Active Copy Status

Information about the currently active copy on each DR is available via entries of the form `ACTIVE_{property}_{DR}` (Table 3.9).

Table 3.9: Active Copy MIB Entries

Entry	**Description**
`ACTIVE_ID_DR{n}`	Copy ID of the active transfer ("None" if idle)
`ACTIVE_STATUS_DR{n}`	Status string of the active transfer
`ACTIVE_BYTES_DR{n}`	Bytes transferred by the active copy
`ACTIVE_PROGRESS_DR{n}`	Percentage progress (e.g., "42%")
`ACTIVE_SPEED_DR{n}`	Current transfer speed (e.g., "12.34MB/s") or "paused"
`ACTIVE_REMAINING_DR{n}`	Estimated time remaining (e.g., "01:23:45") or "unknown"

Example: `ACTIVE_PROGRESS_DR2` returns the percentage progress of the active copy on DR2.

Section 4

Queue Management

This section describes the persistent queue system used by SmartCopy to manage copy and delete operations.

4.1 Queue Architecture

SmartCopy maintains one logical queue per DR (DR1–DR5), all backed by a single shared SQLite database file (`smartcopy.db`). Each queue operates as a FIFO (First In, First Out) structure with the following properties:

- **Persistence**: All queue items are persisted to disk via SQLite, surviving system restarts and crashes
- **Thread Safety**: Database access is protected by a reentrant lock (`threading.RLock`) shared across all queue instances
- **WAL Mode**: The database uses Write-Ahead Logging for improved concurrent read/write performance
- **Transactional**: All queue mutations use `IMMEDIATE` transactions for atomicity
- **Integrity Recovery**: On startup, any items left in "processing" state (from a crash) are automatically reset to "pending"

4.2 Database Schema

The queue database schema is shown in Table 4.1.

Two indexes are maintained for query performance:

- `idx_queue_status` on `(queue_name, status)` — used for queue operations and statistics
- `idx_created` on `(created_at)` — used for time-based queries

4.3 Item Status Lifecycle

Queue items progress through the statuses shown in Table 4.2.

Table 4.1: Queue Database Schema (`queue_items` Table)

Column	Type	Description
`id`	INTEGER	Auto-incrementing primary key
`queue_name`	TEXT	Queue identifier (e.g., "DR1", "DR2")
`host`	TEXT	Source hostname
`hostpath`	TEXT	Source file path
`dest`	TEXT	Destination hostname
`destpath`	TEXT	Destination file path (or delete marker)
`filesize`	INTEGER	File size in bytes (default: 0)
`command_id`	INTEGER	Unique copy/delete ID
`retry_count`	INTEGER	Number of retry attempts (default: 0)
`last_try`	REAL	Timestamp of last attempt (default: 0.0)
`status`	TEXT	Item status (default: "pending")
`created_at`	TIMESTAMP	Creation timestamp

Table 4.2: Queue Item Statuses

Status	Description
`pending`	Item is waiting in the queue to be processed
`processing`	Item has been dequeued and is currently being copied/deleted
`completed`	Transfer completed successfully; source file eligible for purging
`failed`	Transfer failed after exhausting all retry attempts or the source file no longer exists

The lifecycle of a queue item is:

1. Item is added via `SCP` or `SRM` with status `pending`
2. When dequeued for processing, status changes to `processing`
3. On completion, the `processing` row is deleted and a new `completed` row is inserted (for successful copies) or a `failed` row (for permanent failures)
4. If a copy fails but retries remain, the item is re-added as `pending` with an incremented `retry_count`

4.4 Retry Logic

Failed copy operations are retried according to the following rules:

- The maximum number of retries is controlled by the `max_retry` configuration parameter (default: 7)
- The minimum wait time between retries is controlled by the `wait_retry` configuration parameter (default: 24 hours)
- If a retry is attempted too soon (within `wait_retry` hours of the last attempt), the item is returned to the end of the queue
- A copy is permanently failed if:
  - The source file no longer exists, or
  - The retry count has reached `max_retry`
- Delete operations (via `SRM`) have their retry count incremented by an additional 100 (on top of the normal increment of 1) to distinguish them from copy retries

4.5 Remote Copy Exclusivity

To prevent network saturation, only one remote copy may be active at any time across all DR queues. In this context, a copy is considered remote when the source and destination hostnames differ (regardless of whether the destination is on the local network or behind a gateway). This is a simpler test than the gateway detection used for bandwidth limiting (Section 5.5). Exclusivity is enforced by a shared `threading.Semaphore`. When a DR queue manager encounters a remote copy item:

1. It attempts to acquire the remote copy lock (non-blocking)
2. If the lock is unavailable, the item is returned to the end of the queue and processing is deferred
3. If the lock is acquired, the remote copy proceeds with the configured bandwidth limit
4. The lock is released when the copy completes (successfully or with failure)

4.6 Delete Markers

Delete operations use special destination path markers to distinguish them from copy operations. When an `SRM` command is received, the destination path is set to one of two marker values:

`smartcopy_queue_delete_this_file` Deferred delete: the file is marked as complete immediately and will be deleted during the next purge cycle.

`smartcopy_now_delete_this_file` Immediate delete (via `-tNOW`): the file is deleted via `rm -f` as soon as the item is processed.

4.7 Purge Operations

The purge process runs daily at approximately 18:00 UTC and operates as follows:

1. Collect all items with status `completed` for the DR
2. Sum the total file sizes
3. If the total exceeds the `purge_size` threshold (default: 1 TB), proceed with deletion
4. For each completed file, execute `sudo rm -f` on the DR via SSH
5. Files that could not be deleted (e.g., due to the queue being paused) are retained for the next purge cycle
6. After purging, purge all `completed` records from the database

Implementation Note

Spectrometer files (paths containing `DROS/Spec`) are only added to the completed list after being successfully copied to the remote archive. This prevents data loss if a spectrometer file is copied locally but not yet archived remotely.

Section 5

Copy Engine

This section describes the `InterruptibleCopy` class that provides the file transfer capabilities of SmartCopy.

5.1 Overview

The copy engine wraps `rsync` to provide reliable, resumable file transfers with real-time progress monitoring. Each active copy operation is managed by an `InterruptibleCopy` instance that runs the `rsync` process in a background thread and monitors its output.

5.2 Copy Modes

SmartCopy supports four copy modes, determined by the relationship between the source host and destination host (Table 5.1).

Table 5.1: Copy Modes

Mode	**Condition**	**Mechanism**
Local to local	source host = “”, dest host = “”	`rsync` locally, no SSH
Local to remote	source host = “”, dest host ≠ “”	`rsync` from local filesystem to remote destination
Remote same-host	source host = dest host ≠ “”	SSH to DR, `rsync` locally on the DR
Remote to remote	source host ≠ “”, source host ≠ dest host	SSH to source DR, `rsync` to remote destination

5.2.1 Local to Local Copy

For locally originating copies with a local destination, SmartCopy executes:

```
rsync -avH --append --partial --progress <src> <dest>
```

No SSH wrapper is used.

5.2.2 Local to Remote Copy

For locally originating copies with a remote destination, SmartCopy executes:

```
rsync -avH --append-verify --partial --progress <src> <dest>:<destpath>
```

Local to remote copies use `--append-verify` instead of `--append` for additional integrity checking. The bandwidth limit is applied when the destination is determined to be behind a gateway (see Section 5.5).

5.2.3 Remote Same-Host Copy

For copies where the source and destination are on the same DR, SmartCopy executes:

```
ssh -t -t mcsdr@<DR> 'shopt -s huponexit && \
   rsync -avH --append --partial --progress <src> <dest>'
```

The `shopt -s huponexit` ensures the `rsync` process is terminated if the SSH session is disconnected, preventing orphaned processes on the DR.

5.2.4 Remote to Remote Copy

For copies where the source and destination are on different hosts, SmartCopy executes:

```
ssh -t -t mcsdr@<DR> 'shopt -s huponexit && \
   rsync -avH --append-verify --partial --progress \
   [--bwlimit=<limit>m] <src> <dest>:<destpath>'
```

Remote to remote copies use `--append-verify` instead of `--append` for additional integrity checking. The bandwidth limit is applied when the destination is determined to be behind a gateway (see Section 5.5).

5.3 Pause and Resume

Copy operations can be paused and resumed:

Pause The `rsync` subprocess is killed, terminating the transfer. The copy state is preserved and the status is set to "paused".

Resume A new `rsync` process is started with the same parameters. The `--append` (or `--append-verify`) flag ensures the transfer resumes from where it left off, only transferring bytes not already present at the destination.

Resuming a paused copy does not increment the retry counter.

5.4 Destination File Truncation

Before resuming a copy, SmartCopy truncates the last 512 KB of the destination file. This addresses a failure mode where the final block of a partially transferred file may be corrupted due to an interruption during write. Because local and same-host copies use `rsync --append` (which does not verify existing data), truncation ensures the corrupted tail is discarded before appending resumes.

The truncation is performed via:

```
truncate -c -s -512K <destination_file>
```

This operation is applied only for local and same-host copies. Remote copies use `rsync --append-verify`, which checksums existing data, so truncation is not needed.

5.5 Bandwidth Limiting

For remote copies where the `bw_limit` configuration parameter is non-zero, SmartCopy applies `rsync`'s `--bwlimit` option to limit the transfer rate. Bandwidth limiting is only applied when the destination host is determined to be behind a network gateway, indicating that the traffic must traverse a wide-area link. This gateway-based definition of "remote" is stricter than the hostname comparison used for copy exclusivity (Section 4.5): a copy to a different host on the local network is exclusive but not bandwidth-limited.

5.5.1 Gateway Detection

Before starting a copy, SmartCopy resolves the destination hostname to an IP address and queries the kernel routing table using `ip route get`. If the route to the destination includes a gateway hop (indicated by `via` in the output), the destination is considered remote and bandwidth limiting is applied.

For locally originating copies, the route check runs on the local host:

```
ip route get <dest_ip>
```

For DR-originating copies, the route check runs on the DR via SSH:

```
ssh -t -t mcsdr@<DR> 'ip route get <dest_ip>'
```

If DNS resolution fails or the route check cannot be performed, the destination is assumed to be remote and bandwidth limiting is applied.

5.6 Progress Monitoring

The copy engine monitors `rsync` output in real time by parsing the progress line format:

```
<bytes_transferred> <percent>% <speed> <time_remaining>
```

The following progress metrics are extracted:

- **Bytes Transferred**: Total bytes written to the destination
- **Progress**: Percentage complete (e.g., "42%")
- **Speed**: Current transfer rate (e.g., "12.34MB/s")
- **Time Remaining**: Estimated time to completion (e.g., "01:23:45")

These values are available via the per-DR `ACTIVE_{property}_DR{n}` MIB entries (Section 3.4).

5.7 Delete Operations

Delete operations use `rm -f` rather than `rsync`:

```
ssh -t -t mcsdr@<DR> 'shopt -s huponexit && sudo rm -f <path>'
```

For locally originating deletes:

```
rm -f <path>
```

5.8 Copy Lifecycle

The lifecycle of a copy operation is:

1. **Creation**: An `InterruptibleCopy` instance is created with the source, destination, ID, retry count, and last try timestamp
2. **Retry Check**: If the last attempt was less than `wait_retry` hours ago (default: 24), the copy sets its status to "error: too soon to retry" and does not start
3. **Start**: The `rsync` process is launched in a background thread
4. **Monitoring**: The main thread polls `rsync` stdout/stderr for progress updates
5. **Completion**: When `rsync` exits:
   - Return code 0: status set to "complete"
   - Return code < 0 (killed by signal): status set to "paused"
   - Return code > 0: status set to "error: *stderr output*", and any rsync error messages from stdout are prepended
6. **Post-Processing**: The queue manager handles the result (log completion, schedule retry, or record failure)

Section 6

Station Monitoring

This section describes how SmartCopy monitors station activity to coordinate copy operations with DR recording schedules.

6.1 MCS Log Monitoring

SmartCopy monitors the MCS log file (`/home/op1/MCS/sch/mselog.txt`) using `tail -F` to track DR state changes in real time. The monitor thread runs continuously and processes log entries as they appear.

6.2 Log Entry Parsing

Each log entry is parsed to extract the fields listed in Table 6.1.

Table 6.1: Parsed Log Entry Fields

Field Index	Name	Description
5	Reference ID	MCS command reference number
6	Status Code	MCS command send status (2 = sent, 3 = responded, 8 = dead)
7	Subsystem	Target subsystem identifier
8	Command	MCS command name
9	Data	Command arguments or response data

Only entries for DR subsystems (subsystem names starting with "DR") are processed.

6.3 State Determination

The monitor determines DR busy/idle state based on the command status code and command type (Table 6.2).

Table 6.2: DR State Determination Rules

Status Code	Command	Condition	DR State
3	`SHT`, `REC`, `SPC`	Responded	Busy
3	`INI`, `STP`	Responded	Idle
3	`RPT` (`OP-TYPE`)	Response starts with "Idle"	Idle
3	`RPT` (`OP-TYPE`)	Response does not start with "Idle"	Busy
3	`RPT` (`SUMMARY`)	Response is not "NORMAL"	Busy
3	`RPT` (`SUMMARY`)	Response is "NORMAL"	(no change)
8	Any	Subsystem dead	Busy

Implementation Note

For `RPT` commands, the monitor uses a 64-entry cache to correlate responses (status code 3) with their original requests (status code 2). This is necessary because the response data alone does not indicate which MIB entry was queried.

6.4 State Change Propagation

When a DR state change is detected, the monitor notifies the `SmartCopy` controller, which takes the following actions:

DR becomes idle If the global inhibit flag for the DR is not set, the DR's copy queue is automatically resumed.

DR becomes busy The DR's copy queue is paused, interrupting any active copy.

This ensures that copy operations do not interfere with active recording sessions.

6.5 Initialization Behavior

When the station monitor is started (or restarted during `INI`), all DRs are initially assumed to be busy. Their copy queues are paused until the monitor observes log entries indicating an idle state. This conservative approach prevents copies from starting before the station state is known.

Section 7

Status and Error Handling

This section describes SmartCopy's system states, state transitions, and error reporting mechanisms.

7.1 System States

SmartCopy reports its operational status via the `SUMMARY` MIB entry as a 7-character string (Table 7.1).

Table 7.1: System Status Values

Status	Description
`BOOTING`	System is initializing (INI in progress)
`NORMAL`	System operating normally
`SHUTDWN`	System has been shut down

7.2 State Transitions

The state transition diagram is shown in Figure 7.1.

7.2.1 State Transition Events

- **BOOTING** → **NORMAL**: INI sequence completes successfully; all threads started
- **NORMAL** → **SHUTDWN**: SHT command received; state transitions immediately, then all threads stopped
- **SHUTDWN** → **BOOTING**: INI command received; initialization begins

7.3 Command Exit Codes

Commands return the exit codes listed in Table 7.2 to indicate success or failure reason.

```
+----------+
| BOOTING  |<----+
+----------+     |
     |           |
INI complete     |
     |           |
     v           |
+----------+     |
| NORMAL   |     |
+----------+     |
     |           |
SHT command      |
     |           |
     v           |
+----------+     |
| SHUTDWN  |-----+
+----------+  INI command
```

Figure 7.1: State Transition Diagram

Table 7.2: Command Exit Codes

Code	Description
0x00	Process accepted without error
0x01	Invalid arguments to command (e.g., unknown DR name or unknown copy ID)
0x02	Other error running command (e.g., error queuing copy or delete)
0x03	Blocking operation in progress (INI or SHT is active)
0x04	Subsystem needs to be initialized (system is in SHUTDWN state)

7.4 Error Response Format

When a command is rejected, the response data contains the exit code and explanation:

Listing 7.1: Error Response Format

```
0x<CODE>! <LASTLOG MESSAGE>
```

Example responses:

- `0x01! SCP: Invalid arguments to command - unknown data recorder DR7`
- `0x03! INI: Blocking operation in progress - SHT is active and blocking`
- `0x04! SCP: Subsystem needs to be initialized`

7.5 Process Locking

SmartCopy maintains a process list to prevent conflicting operations:

- **INI**: Blocks other INI and SHT commands
- **SHT**: Blocks other INI and SHT commands

The `activeProcess` list tracks currently running blocking operations. Data management commands (`SCP`, `SRM`, `PAU`, `RES`, `SCN`) do not block each other but require the system to be in the `NORMAL` state.

7.6 Email Notifications

SmartCopy sends email notifications for failed transfers. The notification system:

- Runs daily alongside the purge operations
- Collects all failed items for each DR queue
- Generates a report listing each failed file with its path, size, and failure reason
- Sends the report via SMTP with TLS using the sender address and credentials from the `email` section of `defaults.json` (resolved via `lwa_auth`)
- The recipient list is currently hardcoded in the source
- After notification, failed records are purged from the database

Section 8

References

8.1 Related Documents

1. MCS Common ICD — Monitor and Control System Interface Control Document
2. DR ICD — Data Recorder Interface Control Document
3. HAL ICD — Heuristic Automation for LWA Interface Control Document

8.2 Software Repository

The SmartCopy source code is maintained in a Git repository. The primary modules are listed in Table 8.1. For detailed usage instructions for the user-facing scripts, see `Documentation/SmartCopy_Scripts.md` in the repository.

Table 8.1: SmartCopy Source Files

File	**Description**
`smart_cmnd.py`	Main command handler and UDP communication
`smartFunctions.py`	SmartCopy class and system control logic
`smartThreads.py`	Station monitoring and per-DR queue processing threads
`smartCommon.py`	Queue, copy engine, and utility classes
`MCS.py`	MCS UDP protocol implementation and reference server
`smartCopy.py`	CLI client for submitting copy commands
`smartDelete.py`	CLI client for submitting delete commands
`smartQuery.py`	CLI client for querying system status
`smartCopyPause.py`	CLI client for pausing queues
`smartCopyResume.py`	CLI client for resuming queues
`smartCopyHelper.py`	Copy helper with LSL metadata parsing
`smartCopyLeo.py`	LEO-specific copy helper
`smartDeleteHelper.py`	Delete helper with DRSU barcode resolution

8.3 Configuration Files

SmartCopy configuration is stored in JSON format in the `defaults.json` file. Credentials and sensitive values use `lwa_auth` references (prefixed with `!LWA_AUTH`) that are resolved at runtime.

Key configuration parameters include:

- `mcs`: Communication settings
  - `message_in_port`: Incoming command port (default: 5050)
  - `message_out_port`: Outgoing response port (default: 5051)
  - `message_ref_port`: ØMQ reference server port (default: 5052)
- `bw_limit`: Remote copy bandwidth limit in MB/s (default: 0.0, disabled)
- `max_retry`: Maximum retry attempts before giving up (default: 7)
- `wait_retry`: Time in hours to wait between retries (default: 24)
- `purge_size`: Data accumulation threshold in TB before source deletion (default: 1.0)
- `email`: Email notification settings
  - `username`: SMTP authentication username (resolved via `lwa_auth`)
  - `password`: SMTP authentication password (resolved via `lwa_auth`)
  - `smtp_server`: SMTP server address (resolved via `lwa_auth`)

8.4 Dependencies

SmartCopy depends on the external packages listed in Table 8.2.

Table 8.2: Software Dependencies

Package	**Description**
`json_minify`	JSON parsing with comment support
`lsl`	LWA Software Library (metadata, SDF handling)
`lwa_auth`	Credential resolution and configuration loading
`netifaces`	Network interface detection for server address
`zmq`	ØMQ messaging for reference server
`git` (GitPython)	Git repository introspection for version tracking

Additionally, SmartCopy uses the following standard library modules:

- `sqlite3` — Persistent queue storage
- `smtplib` / `email` — Error notification emails
- `subprocess` — `rsync` and SSH process management
- `threading` — Concurrent queue processing

8.5 External Tools

SmartCopy requires the system-level tools listed in Table 8.3.

Table 8.3: Required System Tools

Tool	Usage
`rsync`	File transfer engine (local and remote copies)
`ssh`	Remote command execution on DRs (user: `mcsdr`)
`tail`	MCS log file monitoring (`tail -F`)
`du`	File size determination on DRs
`rm`	Source file deletion during purge
`truncate`	Destination file truncation before resuming copies
`ip`	Gateway detection for bandwidth limiting (`ip route get`)

8.6 Service Deployment

SmartCopy is deployed as a `systemd` service (`smart-copy.service`). The service:

- Runs as the `op1` user
- Logs to syslog and a runtime log file
- Working directory: `/home/op1/SmartCopy/`
- Rotates old logs on startup
- Executes: `python3 ./smart_cmnd.py --config defaults.json --debug --log runtime.log`
- Starts after MCS services

Appendix A

Exit and Status Code Reference

This appendix provides a complete reference of all exit codes and status values used by SmartCopy.

A.1 Command Exit Codes

The following exit codes are returned by SmartCopy commands in the response data field when a command is rejected:

Code	Constant	Description
`0x00`	SUCCESS	Process accepted without error
`0x01`	INVALID_ARGS	Invalid arguments to command; unrecognized DR name, unknown copy ID, or unknown MIB entry
`0x02`	OTHER_ERROR	Other error running command; general error condition such as failure to queue a copy or delete operation
`0x03`	BLOCKING_OP	Blocking operation in progress; INI or SHT command is currently executing and prevents new INI/SHT operations
`0x04`	NOT_INITIALIZED	Subsystem needs to be initialized; system is in SHUTDWN state and requires INI before accepting data management commands

A.2 System Status Values

The 7-character status strings reported via `SUMMARY`:

Status	Description
`BOOTING`	System is initializing. Occurs after INI command is received and while initialization sequence is in progress. Data management commands are not accepted.
`NORMAL`	System operating normally. All DR queue managers running, station monitor active, and system ready to accept data management commands.
`SHUTDWN`	System has been shut down. Occurs after SHT command completes. All monitoring and queue processing threads stopped. Pending queue items are preserved in the database. Requires INI to restart.

A.3 Response Code Characters

Single-character response codes in packet byte 38 are listed in Table A.3.

Table A.3: Response Code Characters

Code	Meaning	Description
`A`	Accepted	Command was accepted and executed successfully
`R`	Rejected	Command was rejected; see response data for exit code and reason

A.4 Queue Item Statuses

Status values stored in the `queue_items.status` column of the SQLite database:

Status	Description
`pending`	Item is in the queue waiting to be processed. This is the initial status for all new items and the status items return to after a failed attempt with retries remaining.
`processing`	Item has been dequeued and is currently being copied or deleted. If found on startup (indicating an interrupted operation), the item is automatically reset to `pending`.
`completed`	Transfer completed successfully. The source file is eligible for deletion during the next purge cycle. These records accumulate until the purge threshold is reached.
`failed`	Transfer has permanently failed due to exhausted retries or missing source file. A failure notification will be sent via email and the record will be purged.

A.5 Copy Operation Statuses

Status strings returned by the `InterruptibleCopy` engine and visible in `ACTIVE_STATUS` MIB entries:

Status	Description
`active`	Copy is currently running
`active/started for ...`	Copy has just been started (includes source and destination)
`active/resumed for ...`	Copy has been resumed after a pause (includes source and destination)
`paused`	Copy has been paused (rsync process killed, ready to resume)
`paused for ...`	Copy is paused (includes source and destination)
`queued for ...`	Copy is waiting in the queue (includes source and destination)
`complete`	Copy completed successfully
`canceled`	Copy was canceled via SCN command
`error: ...`	Copy failed (includes error details from rsync stderr)
`error: too soon to retry`	Retry interval has not elapsed since last attempt

A.6 Error Response Format

Error responses are returned with an `R` (Rejected) response code in byte 38 of the response packet. The response data contains:

1. Exit code in hexadecimal (e.g., `0x04`)
2. Exclamation point delimiter
3. Command name and colon separator
4. Descriptive error message from the `LASTLOG` state

Example: `0x01! SCP: Invalid arguments to command - unknown data recorder DR7`